

Lecture 2.

How Computers Execute Programs.

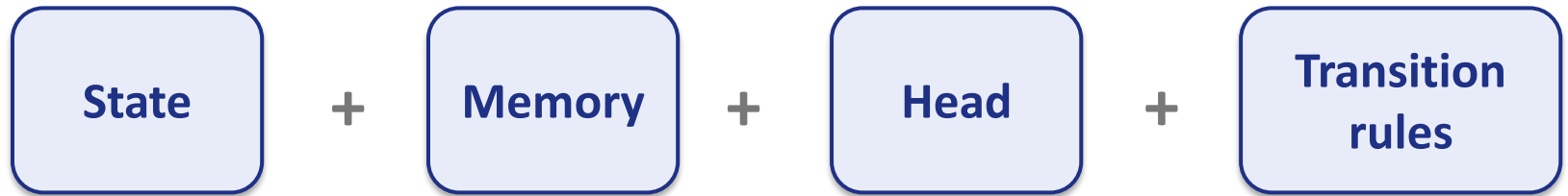
Software Development & Python

Ivan Solomatin

MIPT

The Turing Machine

Main parts



A model of computation tells us how state changes step by step.

The Turing machine is universal — but real computers are built differently.

The Turing Machine

TM vs real computer

Turing machine

Tape: unbounded
abstract memory

One head (limited access)

Abstract machine (no
physical device)

Transitions: one symbol
at a time

Modern computer

RAM: finite physical
memory

RAM: Random Access
Memory

Physical device:
CPU + registers
instruction set

**Different architecture — same idea of
programmable computation.**

Outline

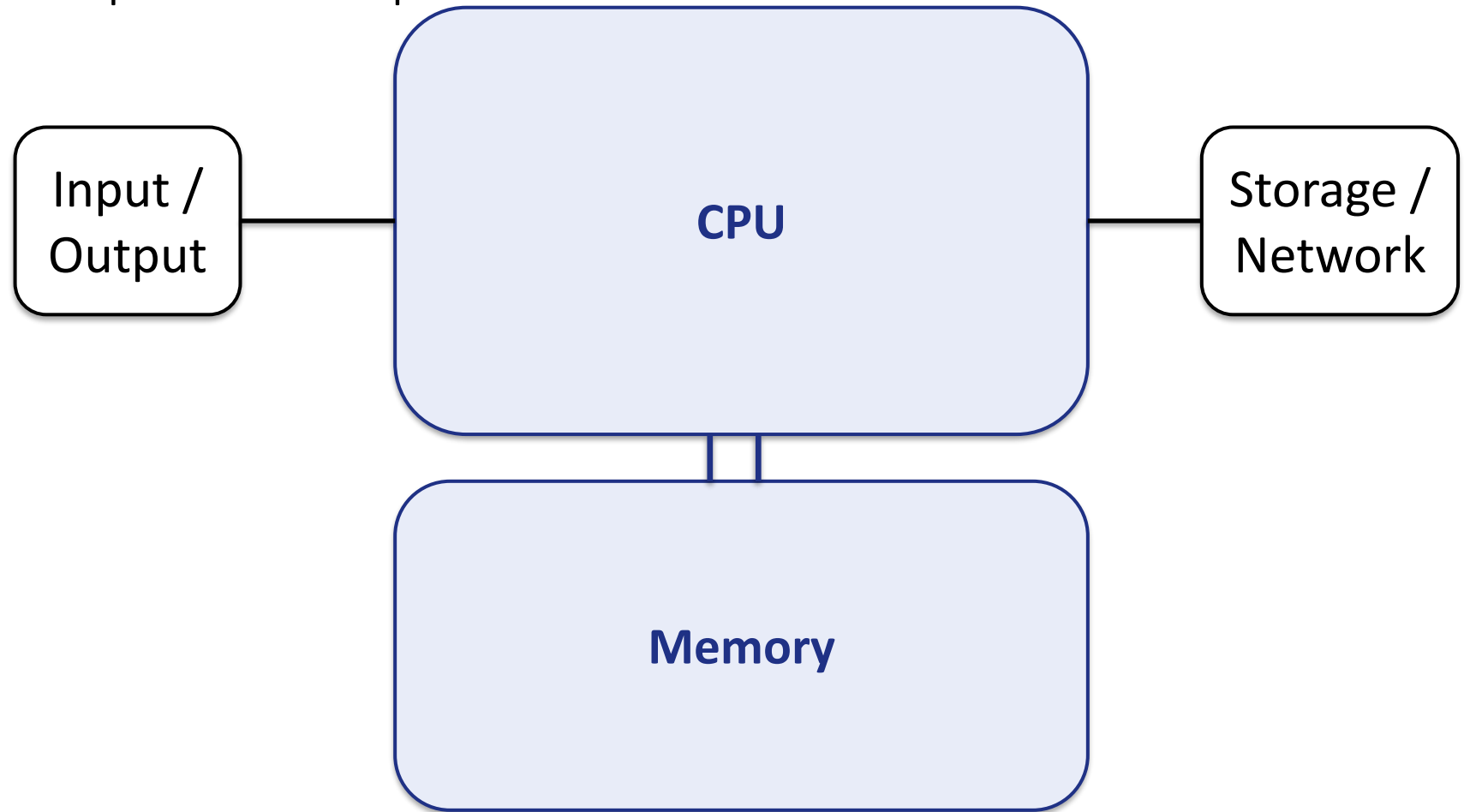
- Von Neumann architecture
- Memory
- CPU
- Fetch → Decode → Execute
- Control flow
- Stack, heap and memory hierarchy
- What modern CPUs actually add

Von Neumann architecture

What is inside a computer?

Von Neumann architecture

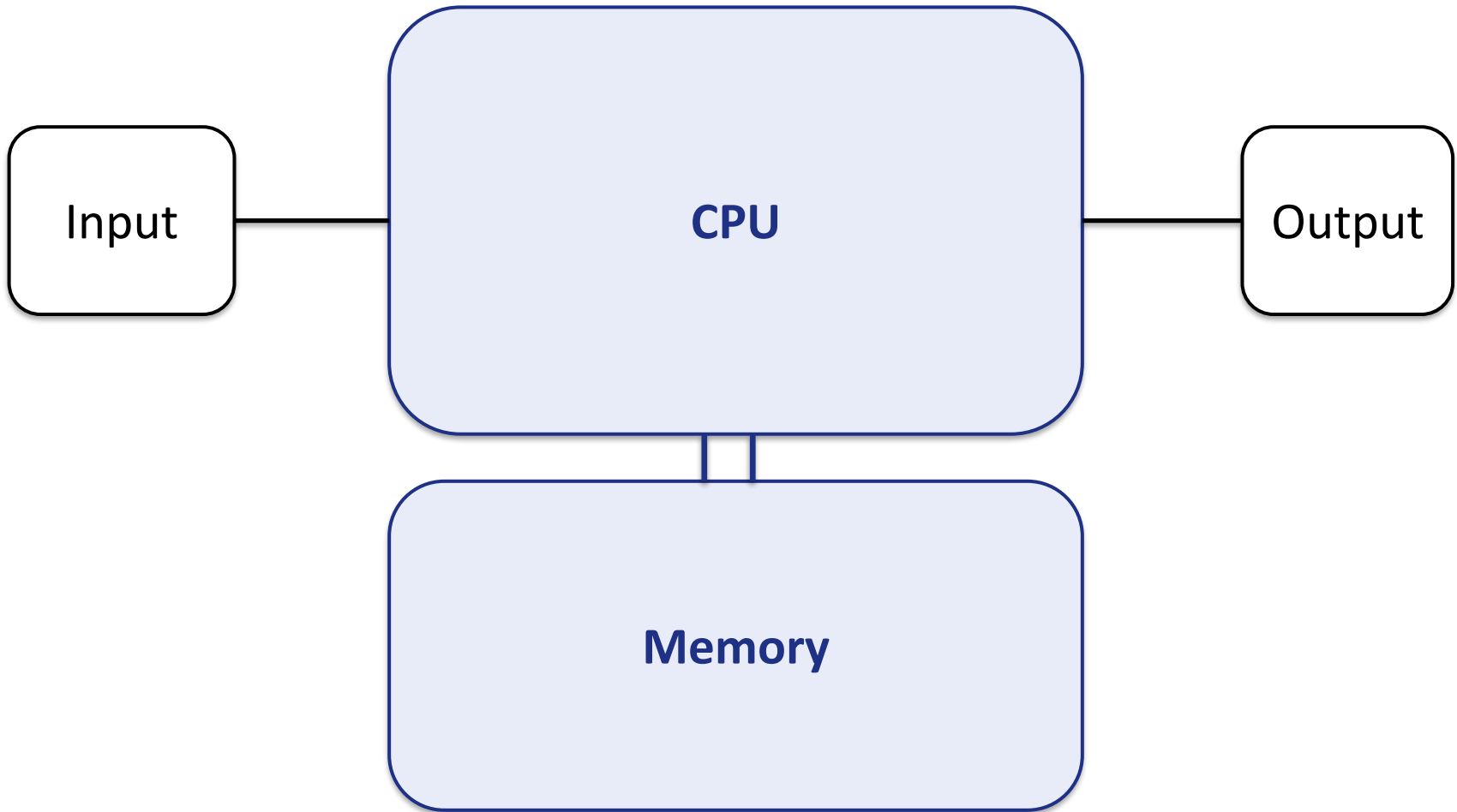
Simplified computer



Most of today can be understood with only these two blocks.⁷

Von Neumann architecture

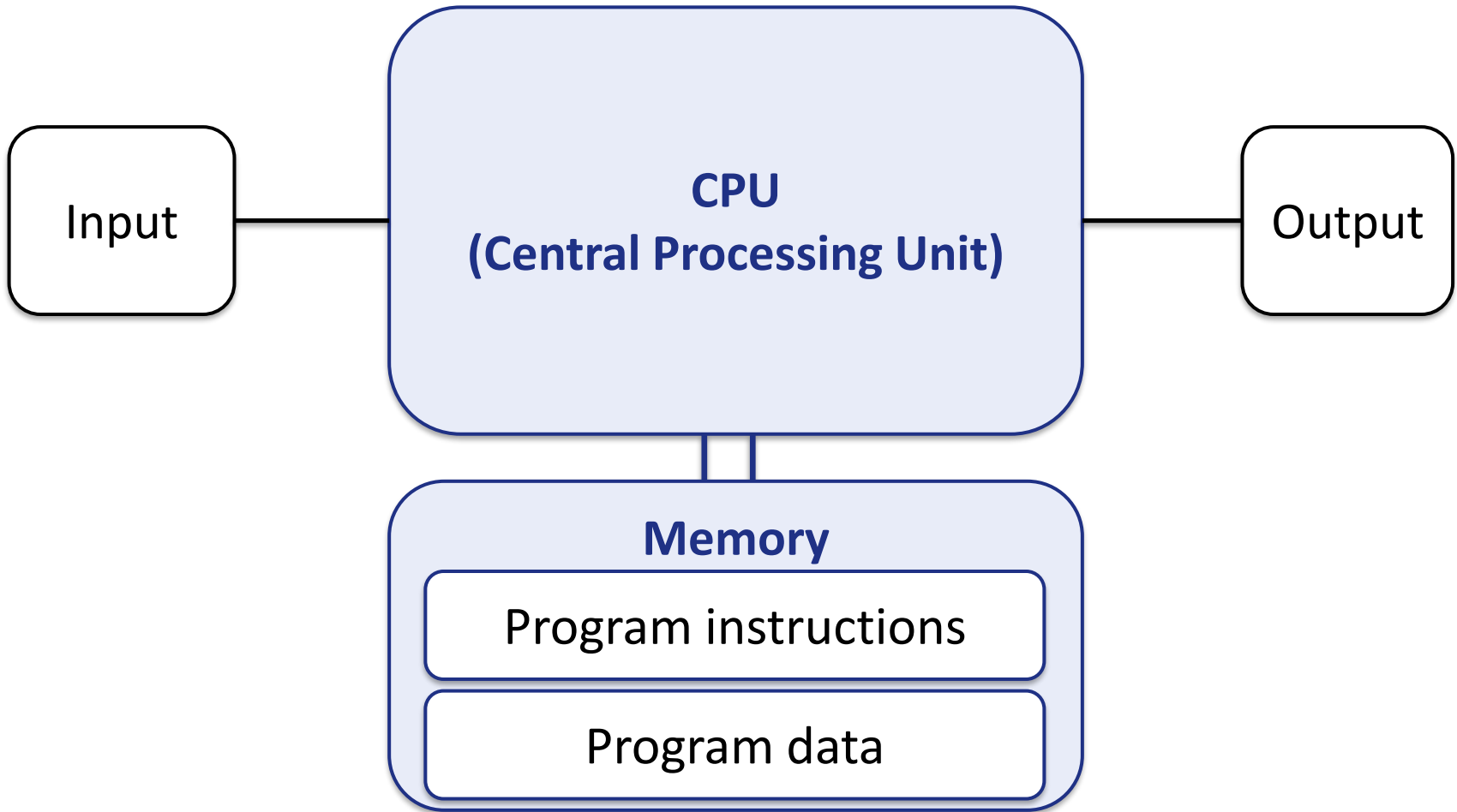
Main scheme



Key idea: program instructions and program data are both stored in memory.

Von Neumann architecture

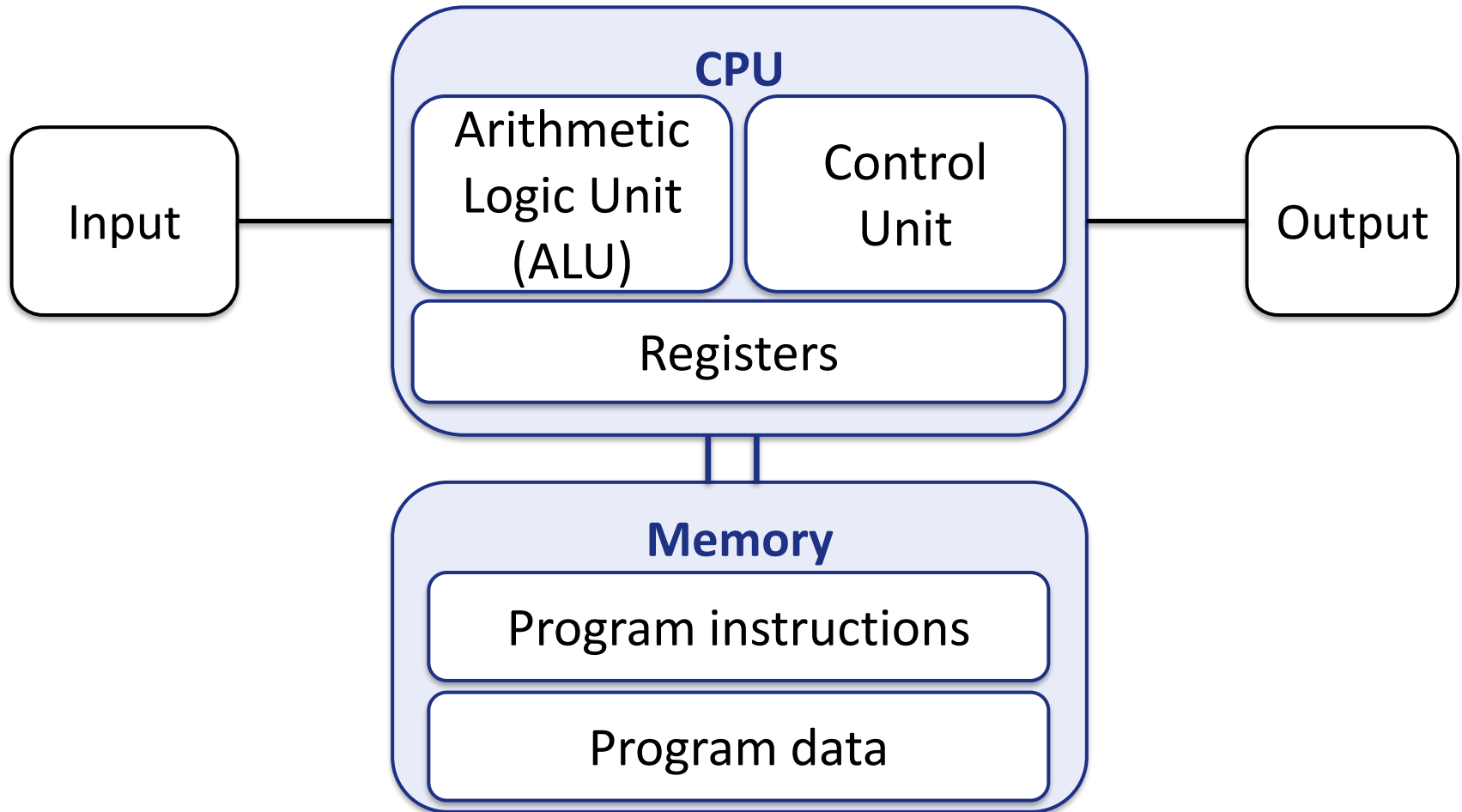
Main scheme



Program is data.

Von Neumann architecture

Main scheme



Memory

Memory



2 GB = 2 000 000 000 Bytes:

4	5	1	6	7	3	6	1	6	8	1	0	2	5	1	1	5	1	6	0
6	0	4	5	1	6	7	3	6	1	6	8	1	0	2	5	1	1	5	1
5	1	1	5	1	6	0	4	5	1	6	7	3	6	1	6	8	1	0	2
6	1	6	8	1	0	2	5	1	1	5	1	6	0	4	5	1	6	7	3
4	5	1	6	7	3	6	1	6	8	1	0	2	5	1	1	5	1	6	0
6	0	4	5	1	6	7	3	6	1	6	8	1	0	2	5	1	1	5	1
5	1	1	5	1	6	0	4	5	1	6	7	3	6	1	6	8	1	0	2
6	1	6	8	1	0	2	5	1	1	5	1	6	0	4	5	1	6	7	3

Memory

Memory as numbered cells

Address		Value
0x0000	—	42
0x0001	—	17
0x0002	—	5
0x0003	—	...

Memory is addressed: an address identifies a location, not a variable name.

Memory

Bits, bytes and addresses

1 bit = 0 or 1

8 bits = 1 byte

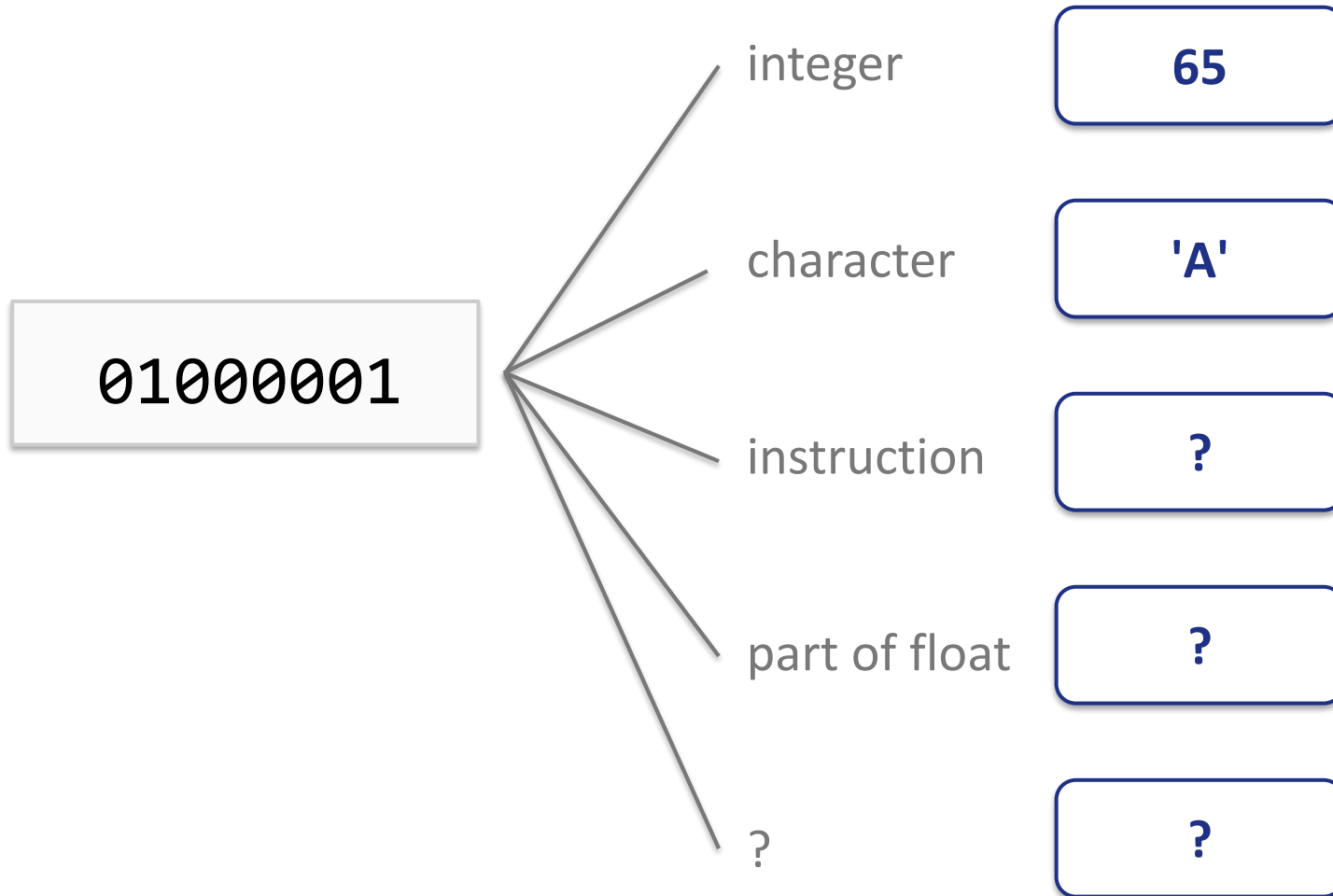
00101010 = 42

Typical memory addresses: 0x0000, 0x0001, 0x0002, ...

0x0000	0x0001	0x0002	0x0003	
00101010	00000010	11111111	00000000	...

Memory

Bits/bytes have no intrinsic type



Interpretation gives bits meaning.

Memory

Overflow

Numbers in memory are finite representations

uint8

00000000

0

11111111

255

255 + 1

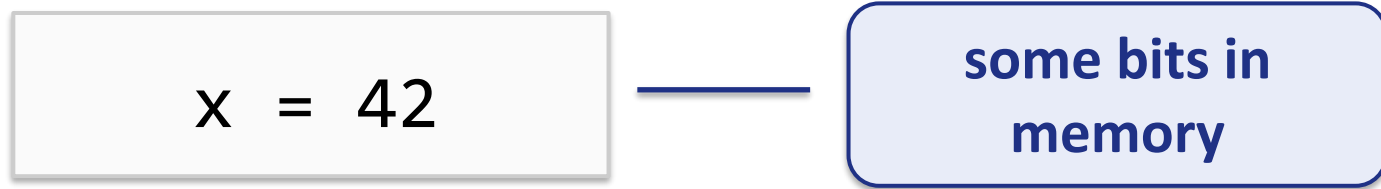


00000000

Overflow is a property of the representation, not of mathematics.

Memory

A variable is an abstraction



High-level languages hide addresses, encodings and many lifetime details.

In Python this abstraction is even stronger: a name is usually bound to an object, not to a fixed typed memory cell.

CPU

CPU

What does a CPU actually do?

read data

write data

perform arithmetic

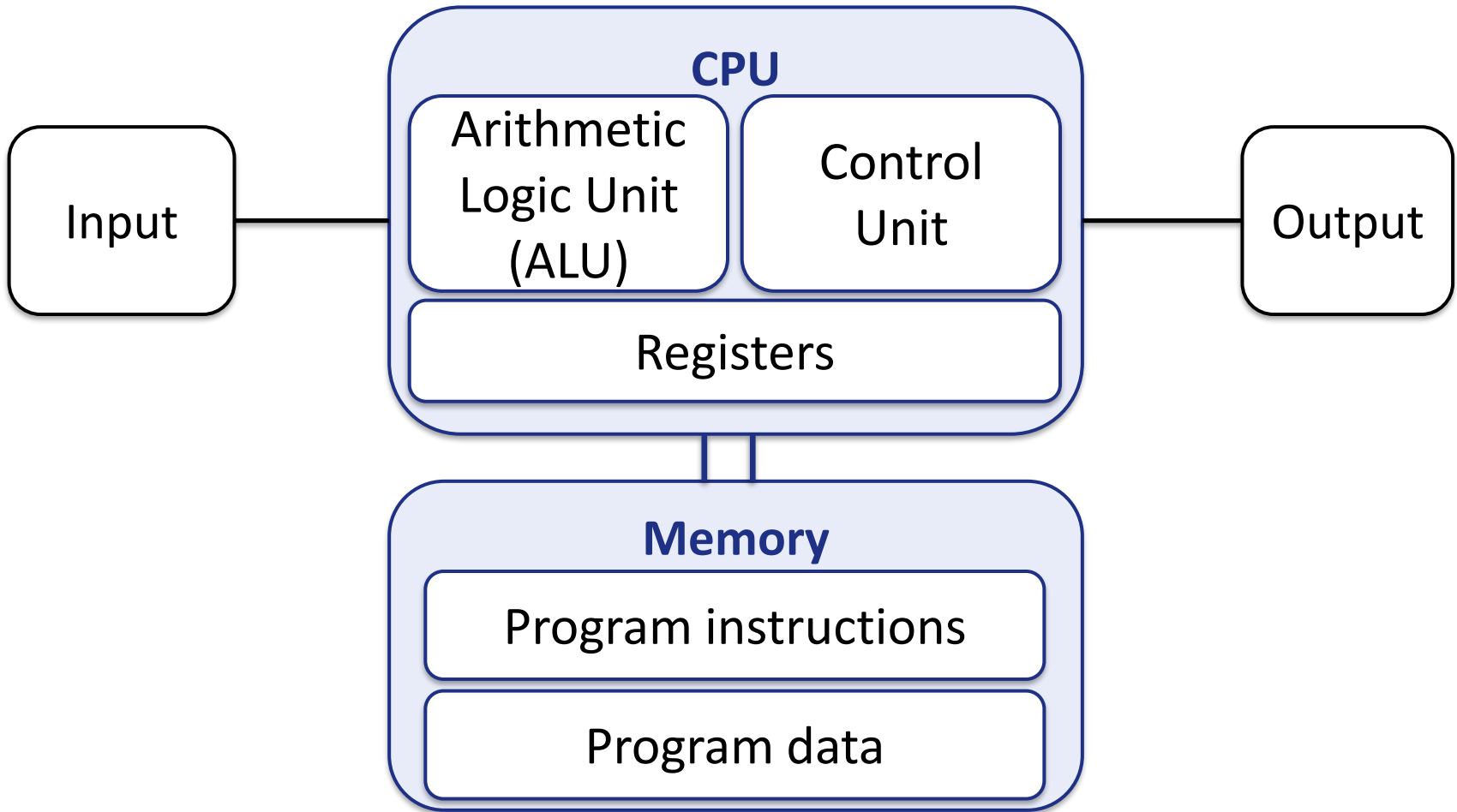
compare values

change control flow

Surprisingly little — but very fast.

CPU

Parts of the CPU

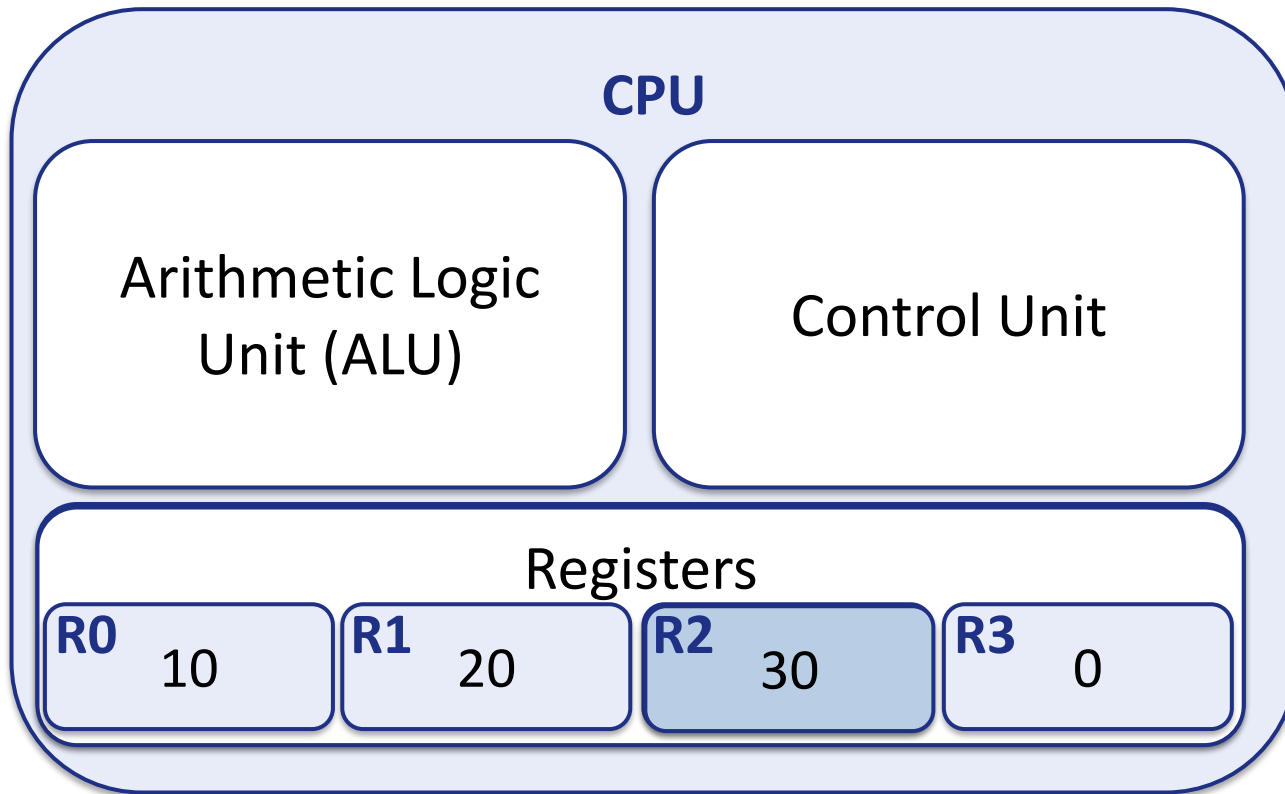


For this course, this simplified model is enough.

CPU

Registers

Registers: tiny and fast storage inside the CPU



ADD R2, R0, R1

$R2 \leftarrow R0 + R1$

$R2 = 30$

CPU

Registers

Here are the main registers of modern i7 CPU (not all of them).

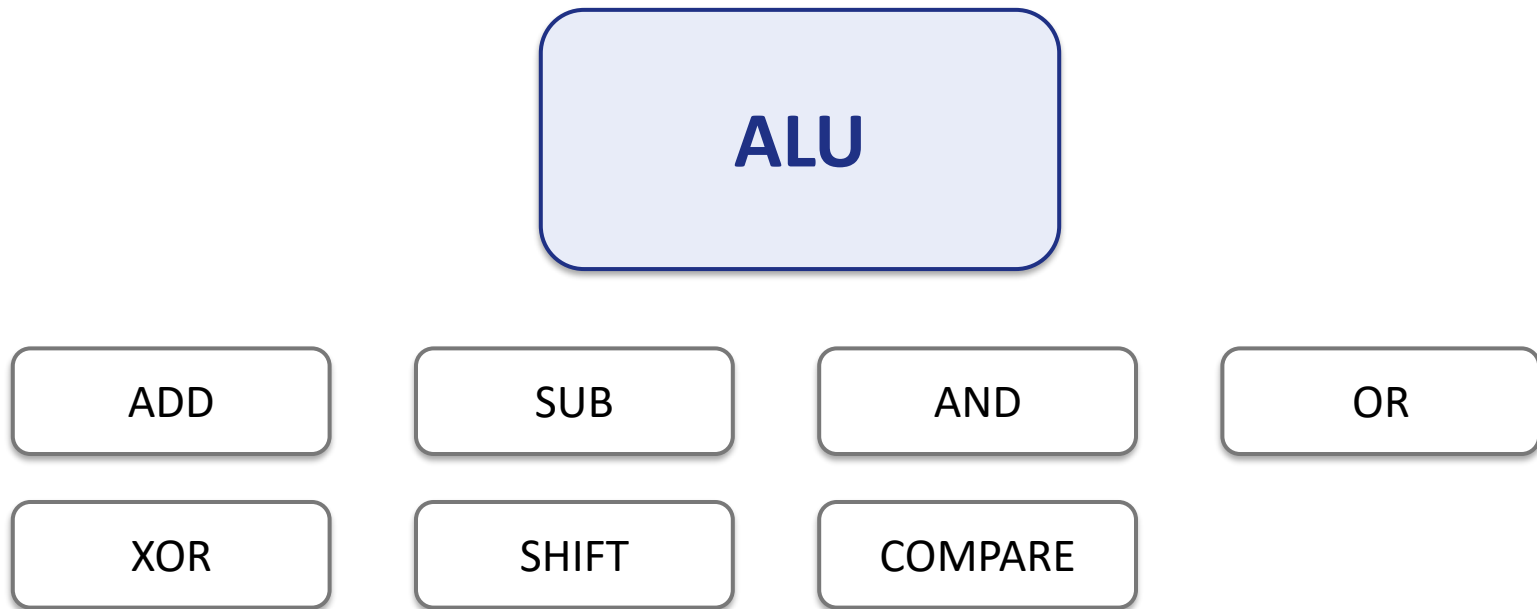
Registers are not for storing the data. They are for active memory CPU works with.

Register Type	Name (64-bit)	Quantity	Size	Purpose / Notes
General-Purpose Registers (GPR)	RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8, R9, R10, R11, R12, R13, R14, R15	8	64 bits	Used for integer arithmetic, memory addressing, and storing pointers.
Instruction Pointer	RIP (Register Instruction Pointer, a.k.a. Program Counter)	1	64 bits	Holds the address of the next instruction to be executed.
Flags Register	RFLAGS	1	64 bits	Holds the status of the last operation (zero, carry, overflow, etc.) and system flags.

CPU

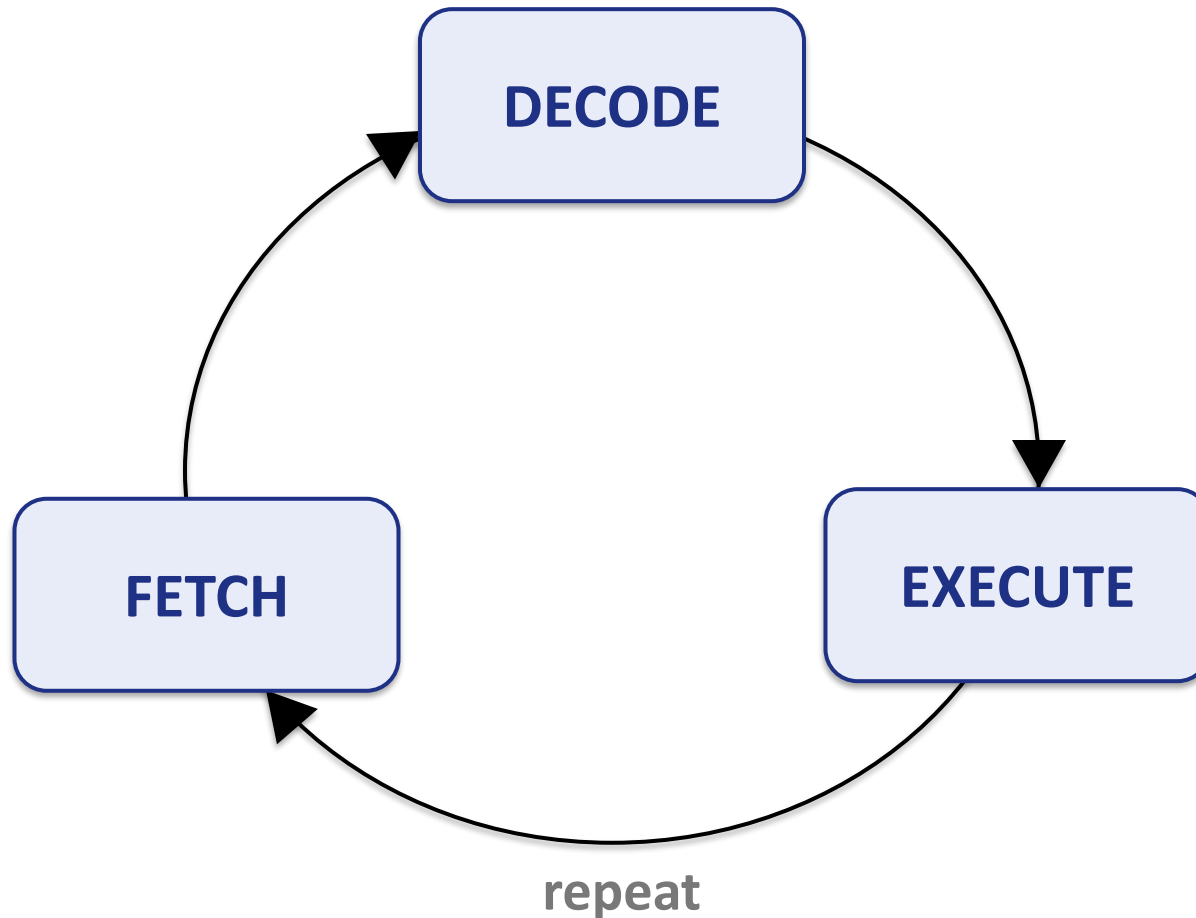
The ALU: arithmetic and logic

Input: register values. Output: a result and/or flags.



CPU

The Control Unit: Managing The Instruction Cycle.



Instruction Set Architecture

The interface between software and CPU

Instruction Set Architecture

The interface between software and CPU

ADD R2, R0, R1

$R2 \leftarrow R0 + R1$

An instruction set architecture (ISA) is an abstract model that defines the programmable interface of the CPU of a computer, defining how software interacts with hardware. It defines:

- Instruction Set
- Data Types
- Registers
- Instruction Formats
- Etc.

**Instruction Set Architecture = contract between
software and hardware**

Instruction Set Architecture

Typical instruction categories

Data movement

LOAD

STORE

MOV

Arithmetic / logic

ADD

SUB

MUL

AND

XOR

Control flow

JMP

JZ

CALL

RET

Instruction Set Architecture

Toy ISA: TinyCPU

Registers

R0

R1

R2

R3

Memory

0

1

2

3

...

Instructions

MOV R, v

LOAD R, a

STORE R, a

ADD R1, R2

SUB R1, R2

JMP a

JZ R, a

HALT

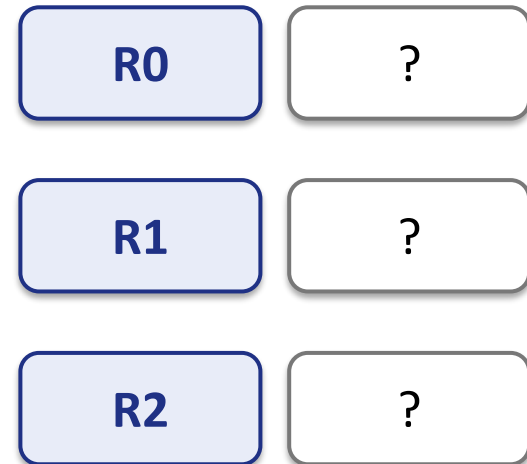
TinyCPU: our first program

TinyCPU: our first program

Toy ISA: TinyCPU

Compute 2 + 3

```
MOV R0, 2  
MOV R1, 3  
ADD R0, R1  
HALT
```



One instruction at a time.

TinyCPU: our first program

Step 1

0	MOV R0, 2
1	MOV R1, 3
2	ADD R0, R1
3	HALT

PC = 0

R0	2
R1	?
R2	?

MOV loads a constant into R0

TinyCPU: our first program

Step 2

0	MOV R0, 2
1	MOV R1, 3
2	ADD R0, R1
3	HALT

PC = 1

R0	2
R1	3
R2	?

MOV loads a constant into R1

TinyCPU: our first program

Step 3

0 MOV R0, 2

1 MOV R1, 3

2 ADD R0, R1

3 HALT

PC = 2

R0

5

R1

3

R2

?

TinyCPU: our first program

How does the CPU know what to execute next?

Program Counter (PC)

0	MOV R0, 2
1	MOV R1, 3
2	ADD R0, R1
3	HALT

PC = 1

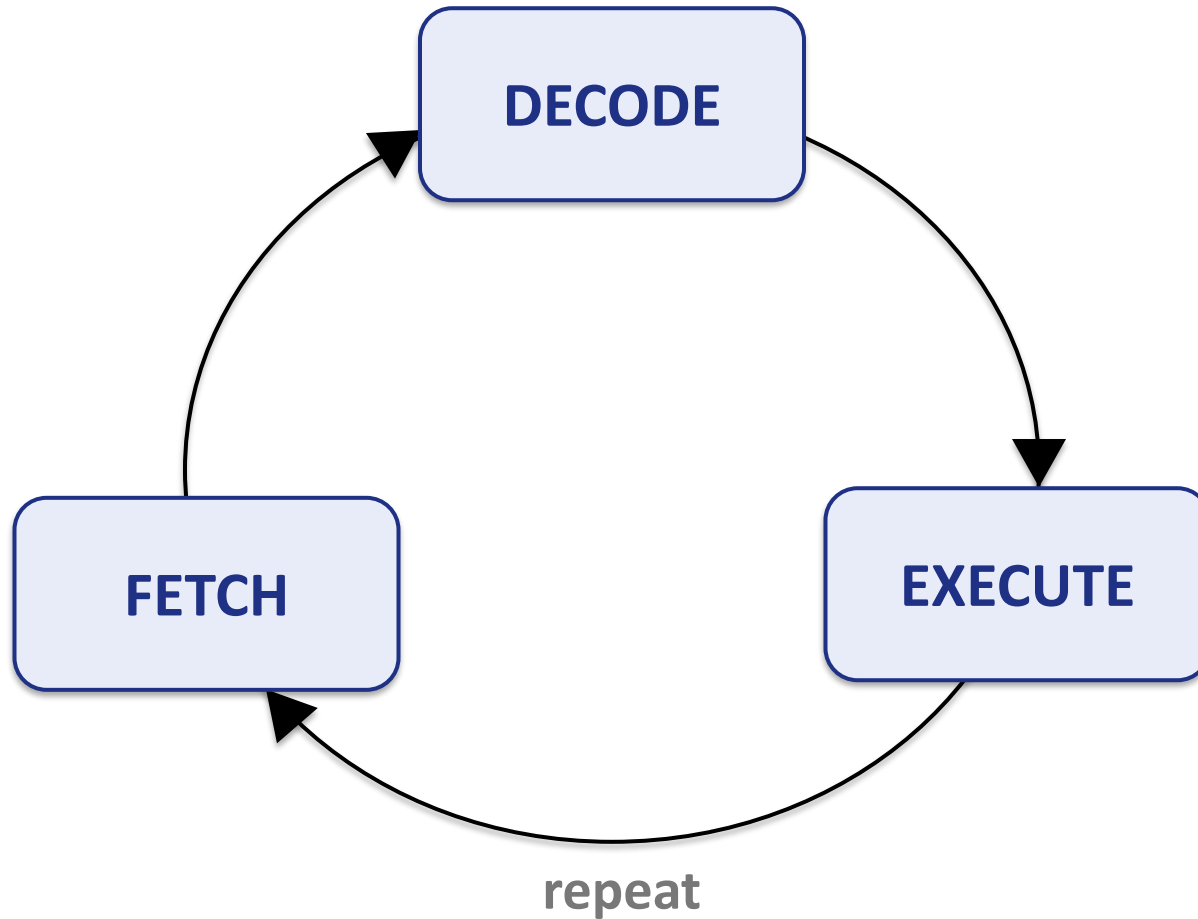
Usually: $PC \leftarrow PC + 1$
Unless a jump changes it.

The instruction cycle

Fetch → Decode → Execute

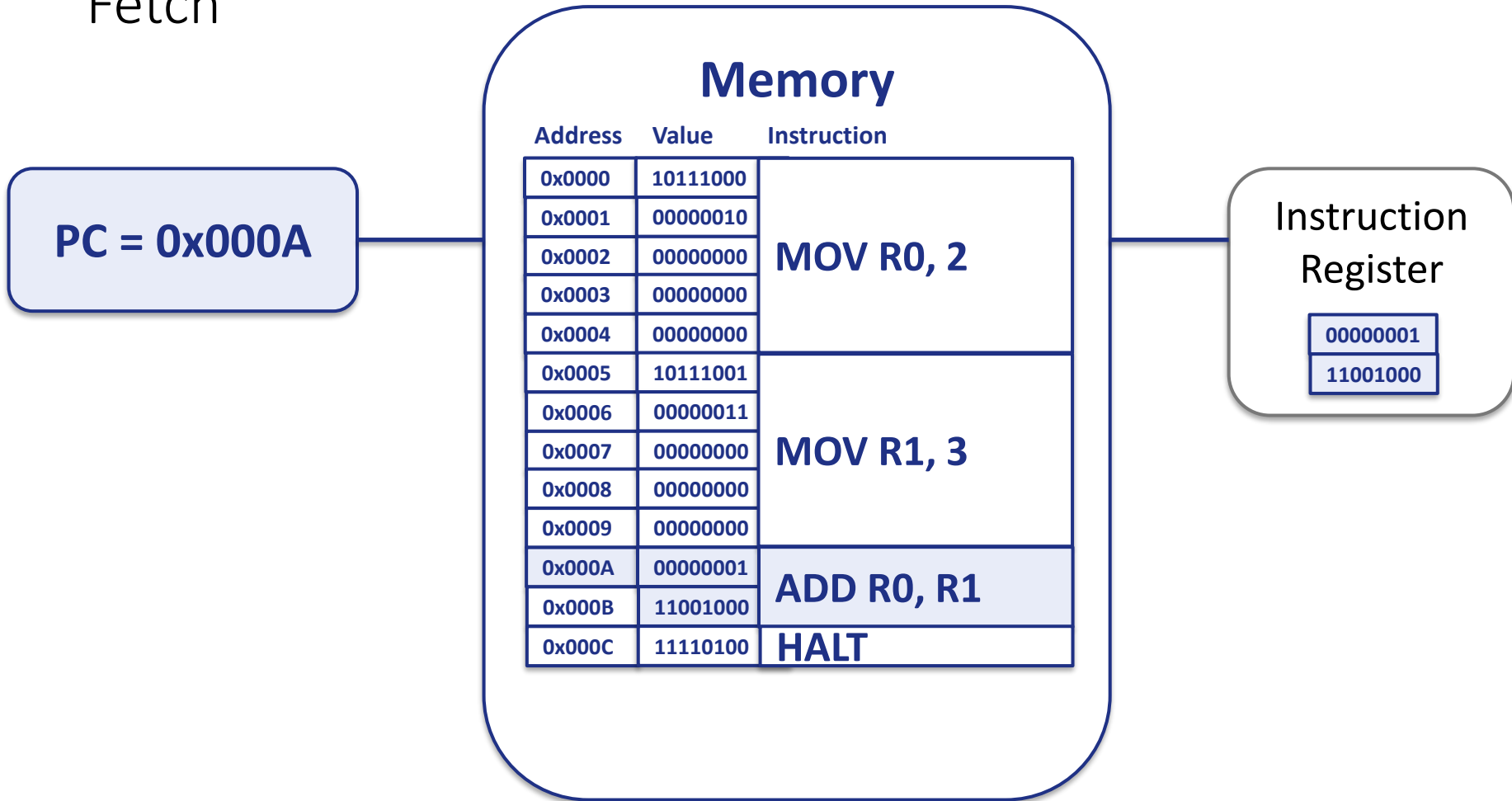
The instruction cycle

Fetch → Decode → Execute



The instruction cycle

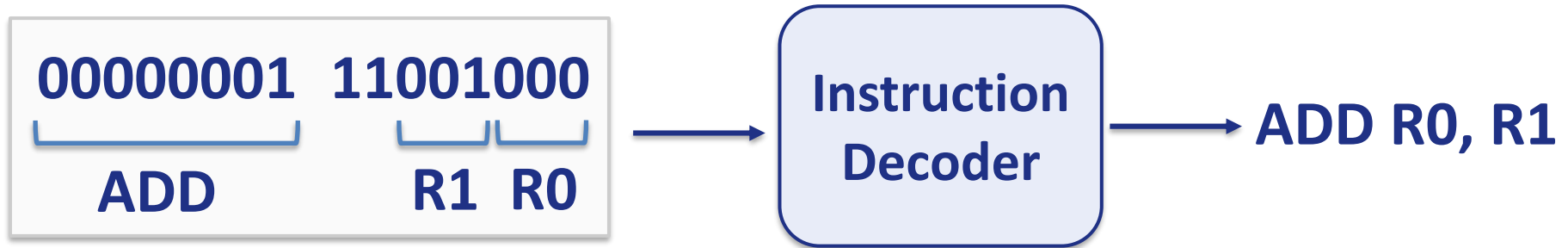
Fetch



**Fetch the instruction located at the address stored in PC.
PC -> Instruction Pointer (IP).**

The instruction cycle

Decode



Machine instructions are encoded as bits.

The CPU interprets fields as opcode + operands.

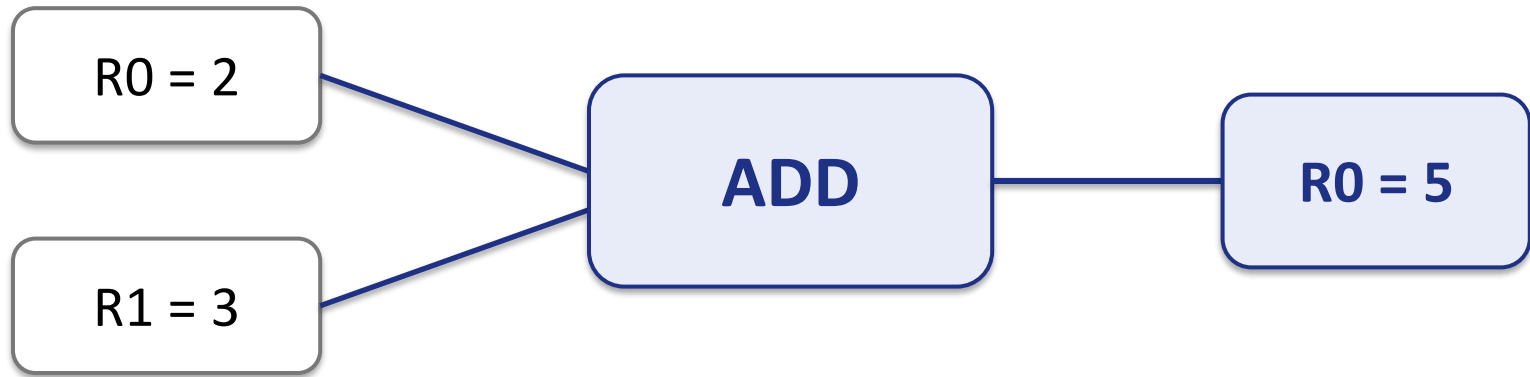
Length of the operation may be variable

(some operations are shorter, some – longer).

PC address is increased by the length of current operation
after execution.

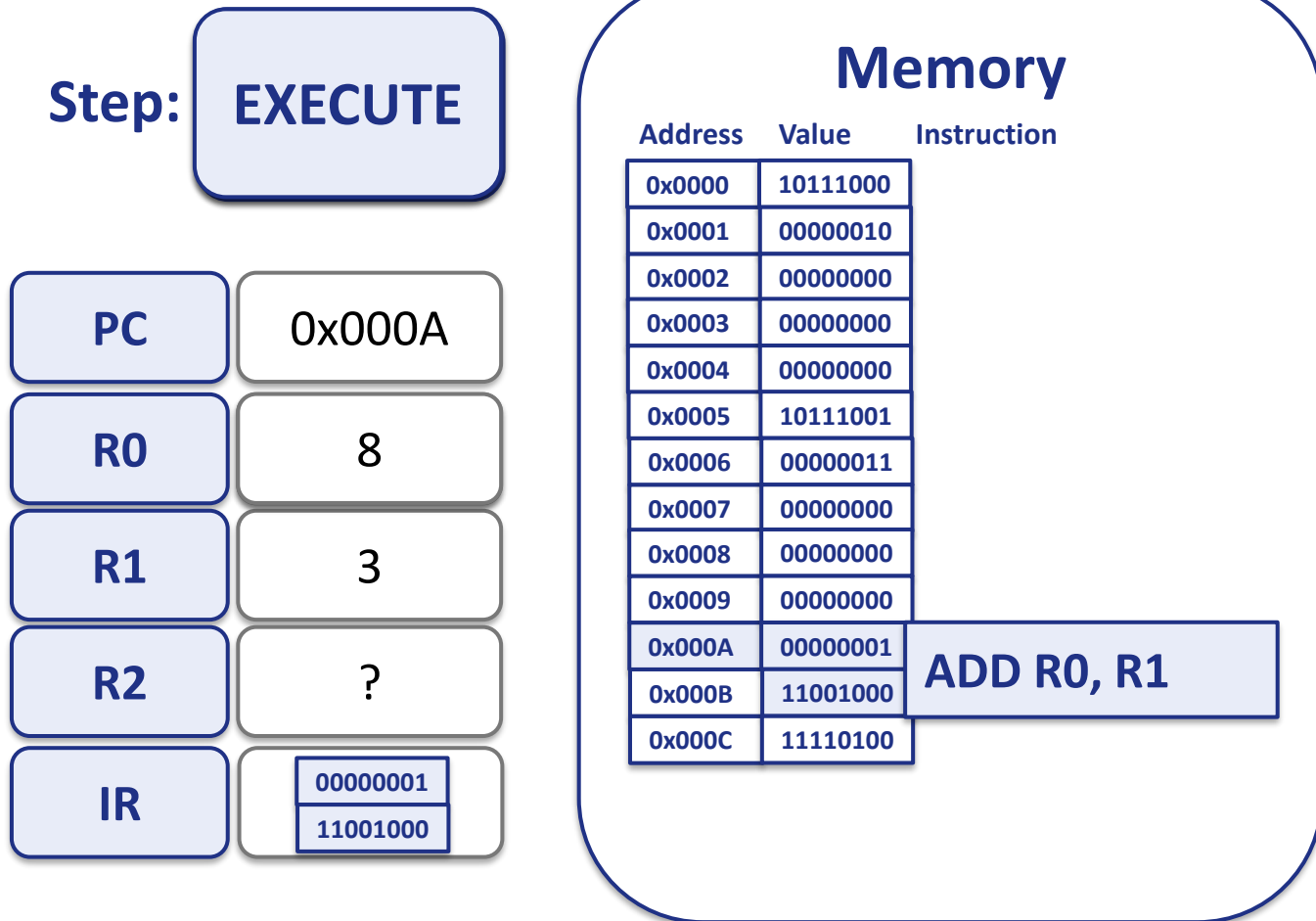
The instruction cycle

Execute



The instruction cycle

Example



Control flow

if, loops and jumps

Control flow

High-level control flow is an abstraction

```
if x == 0:  
    do_A()  
else:  
    do_B()  
do_C()
```



```
CMP R0, 0 #compares and sets ZF if =  
JZ  A      #if ZF is set, jump to A  
...B...  
JMP END  
A:  
...A...  
END:  
...C...
```

The CPU does not know what an “if statement” is.

Control flow

Conditional jump

```
CMP R0, 0  
JZ 10
```

if R0 == 0:

PC = 10



Control flow

Loop

```
while x > 0:  
    x -= 1
```

LOOP:

```
    CMP    R0, 0 ; compare R0 with 0 (R0 - 0, set flags)  
    JLE    END   ; if x <= 0 → END (JLE: Jump if Less or Equal)  
    SUB    R0, 1 ; x = x - 1  
    JMP    LOOP  ; continue the loop
```

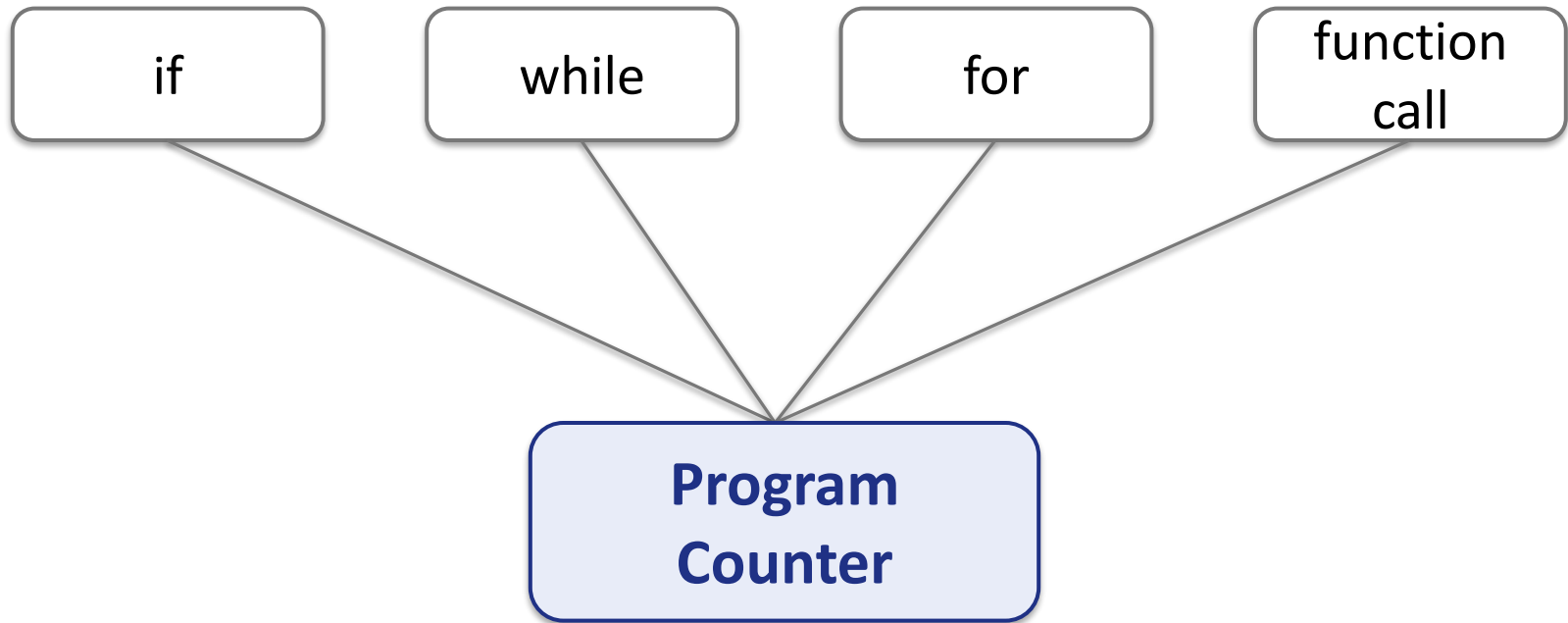
END:

...

The backward JMP is the loop.

Control flow

Control flow = changing the Program Counter



At low level, control flow is mostly about choosing the next instruction.

Functions and the call stack

Functions and the call stack

What happens when we call a function?

```
def square(x):  
    return x * x  
  
y = square(5)
```

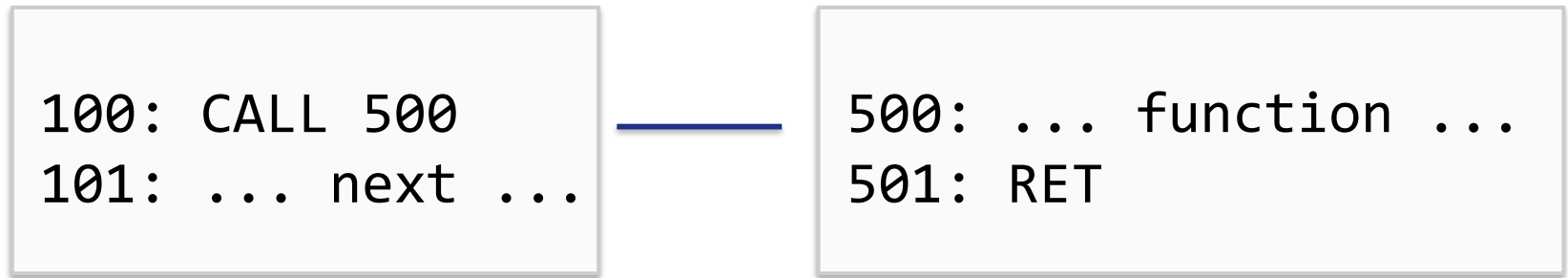
**The CPU has no
“function” object.**

It must:

1. remember where to return
2. jump to function code
3. restore the caller state

Functions and the call stack

CALL and RET



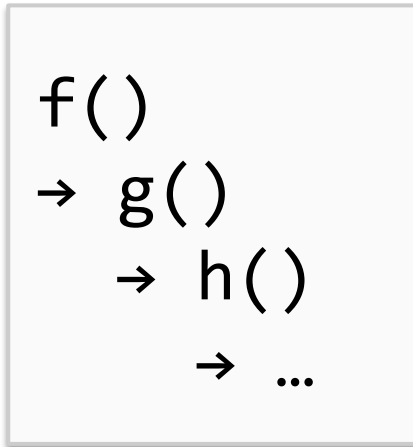
CALL: save return address + jump

RET: restore return address + jump back

Functions and the call stack

Nested calls create a memory problem

We must remember:



where h returns

where g returns

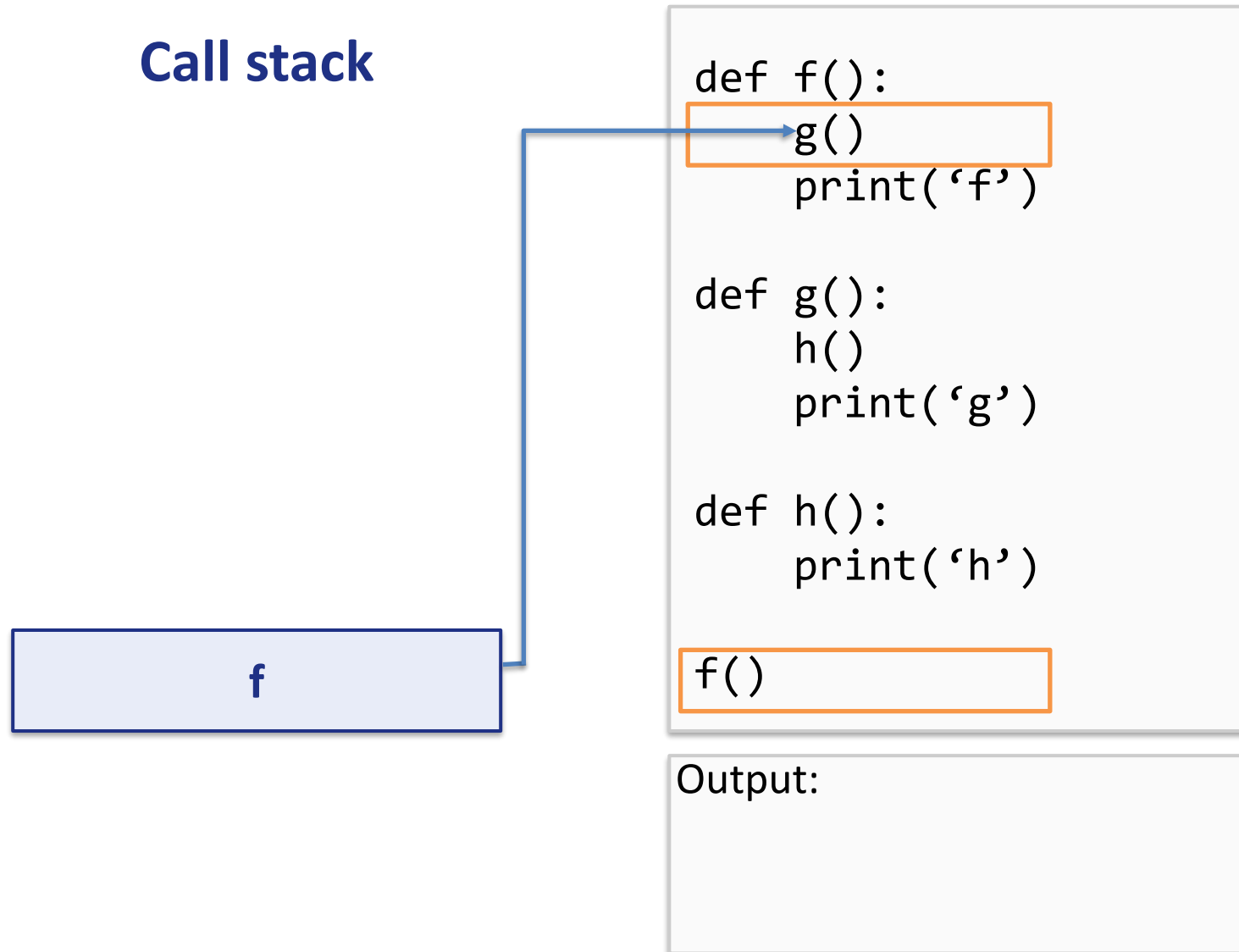
where f returns

...

Last called function returns first → LIFO.

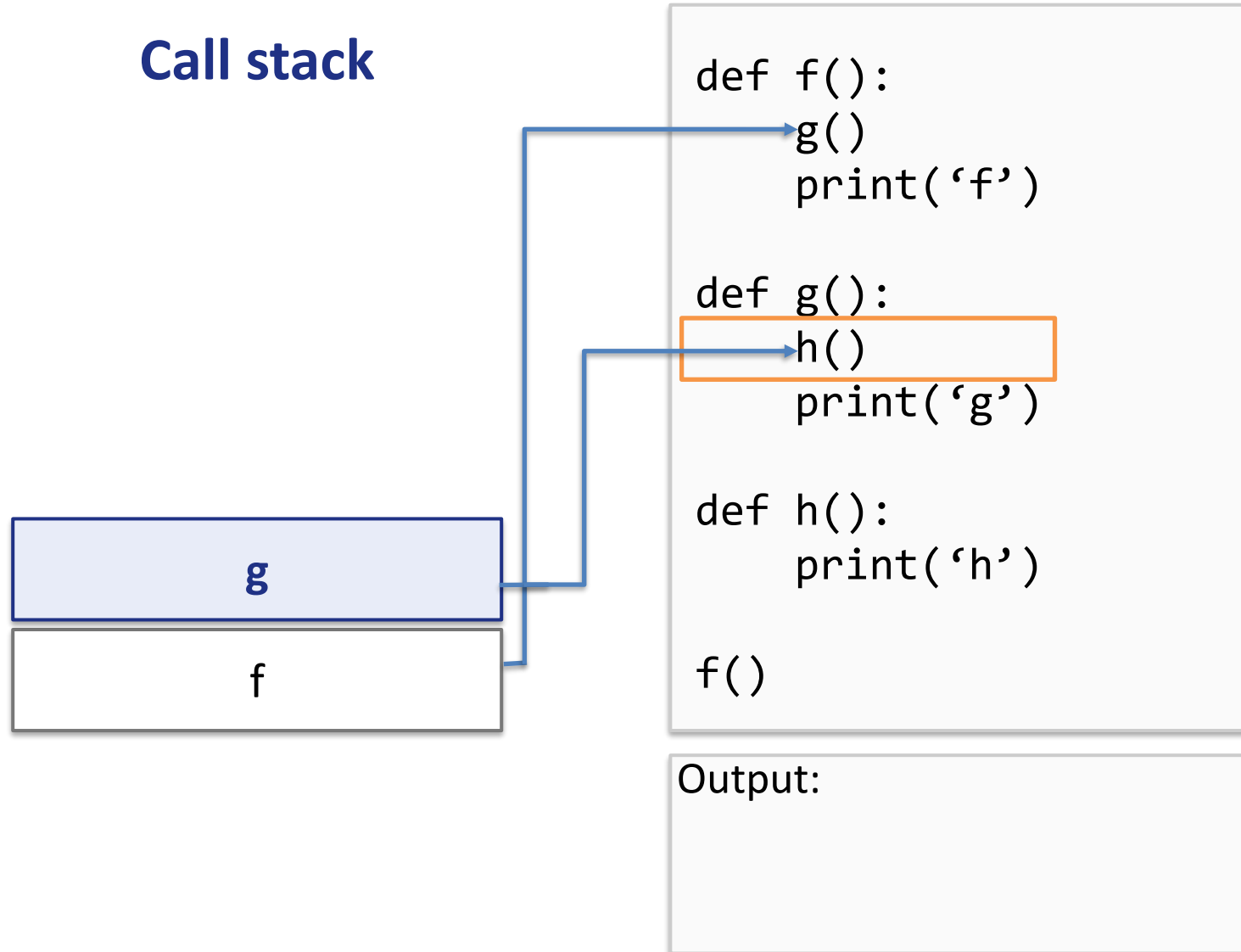
Functions and the call stack

Call stack



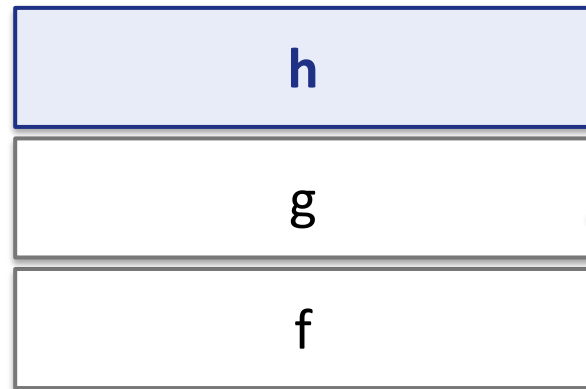
Functions and the call stack

Call stack



Functions and the call stack

Call stack



```
def f():  
    g()  
    print('f')
```

```
def g():  
    h()  
    print('g')
```

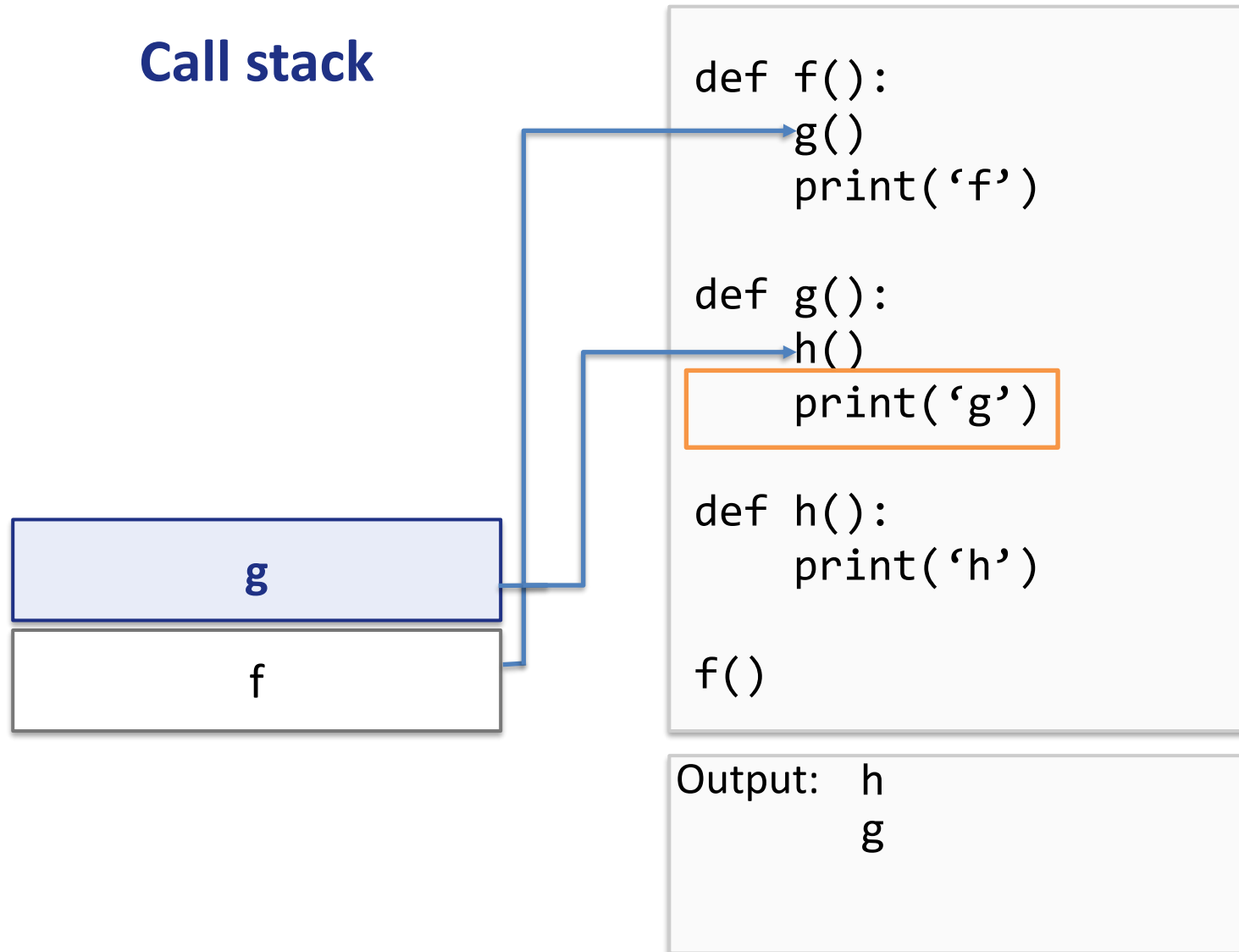
```
def h():  
    print('h')
```

```
f()
```

Output: h

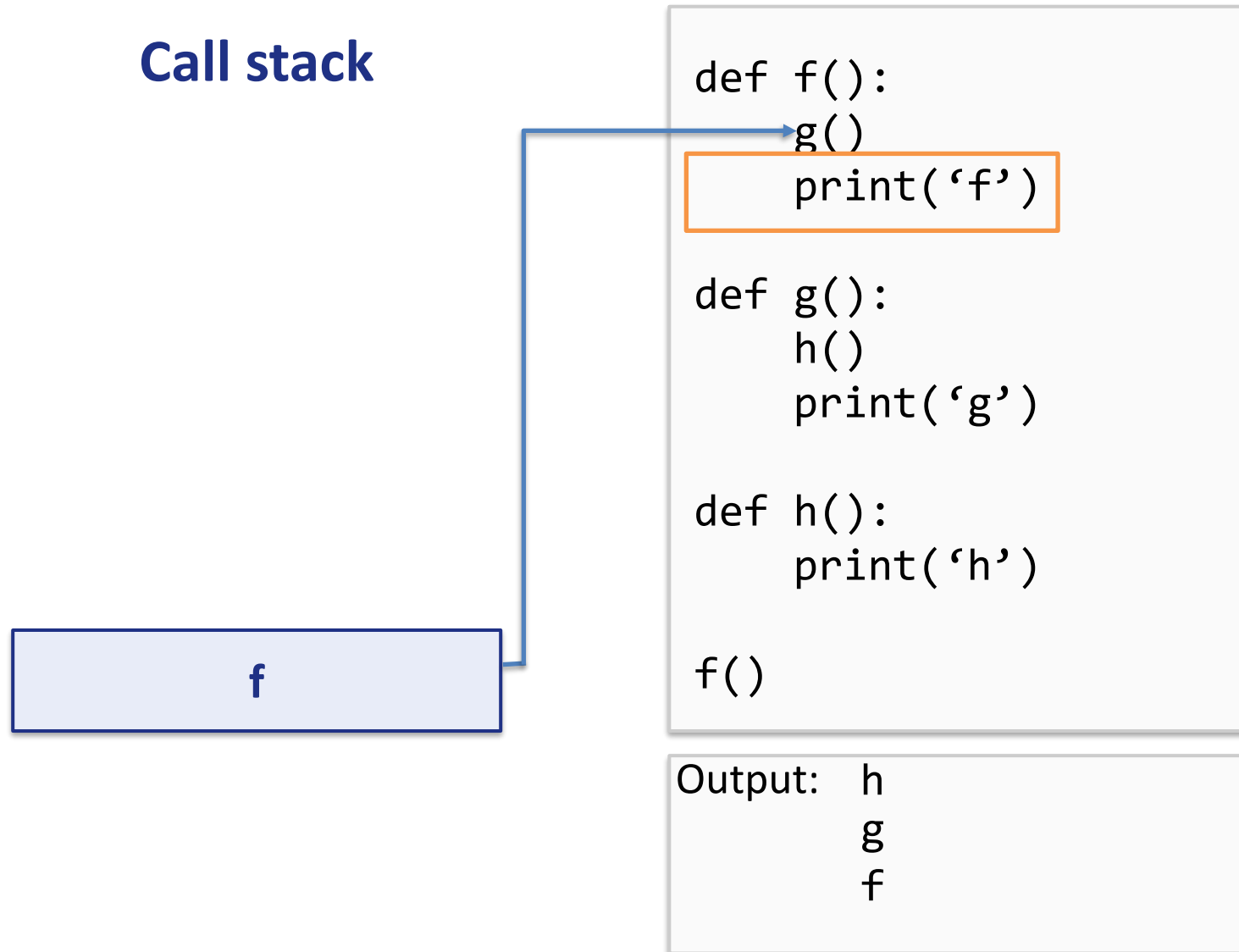
Functions and the call stack

Call stack



Functions and the call stack

Call stack



Functions and the call stack

Recursion

Recursion is just repeated function calls

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n-1)
```

factorial(4)

factorial(3)

factorial(2)

factorial(1)

**Each call gets its own
frame.**

Stack and heap: two different roles

Stack

function calls

automatic lifetime

usually fast

limited size

Heap

dynamic objects

flexible lifetime

allocation is more complex

usually much larger

Useful mental model — not a universal exact map of every runtime.

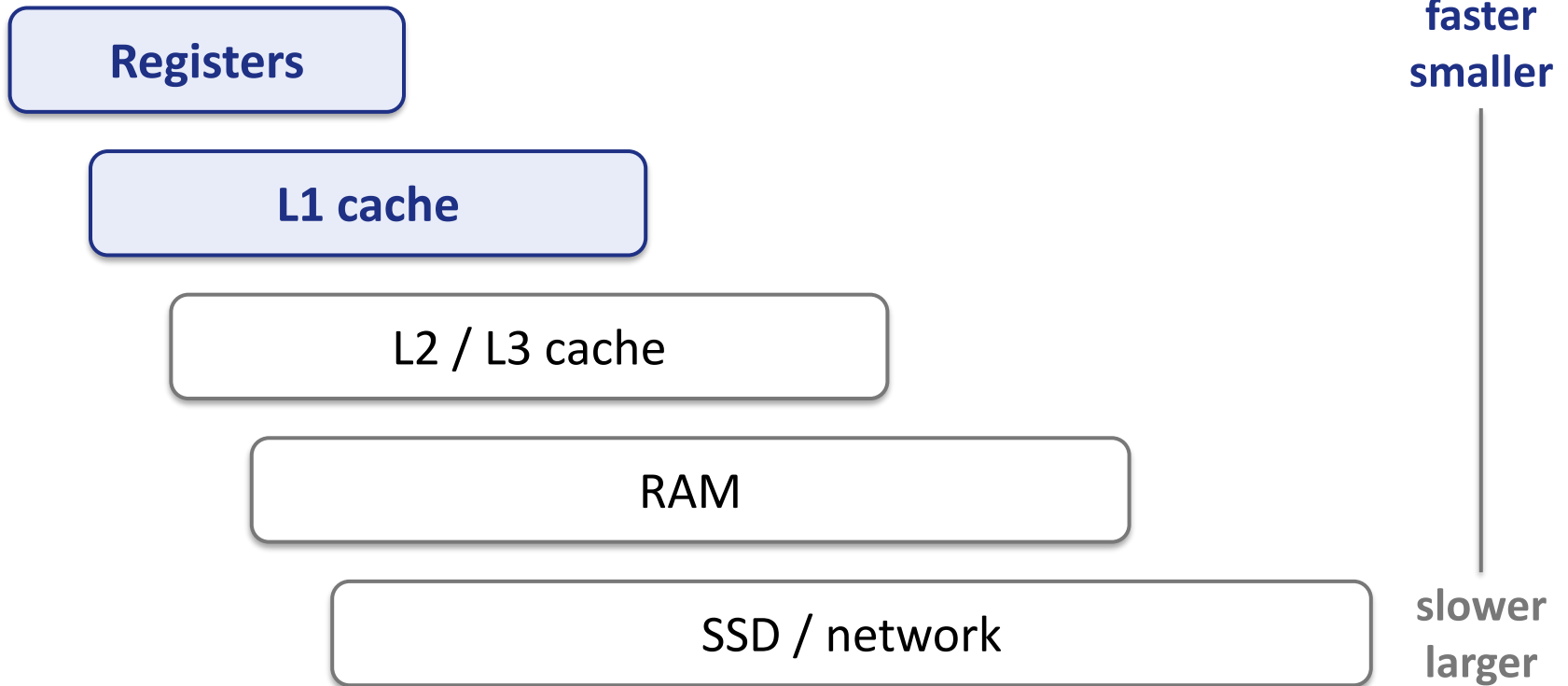
Modern CPUs

Additional mechanisms

Modern CPUs

Caching

Memory is not equally fast



Modern CPUs

Caching

Why software developers should care

```
for row in matrix:  
    for x in row:  
        process(x)
```

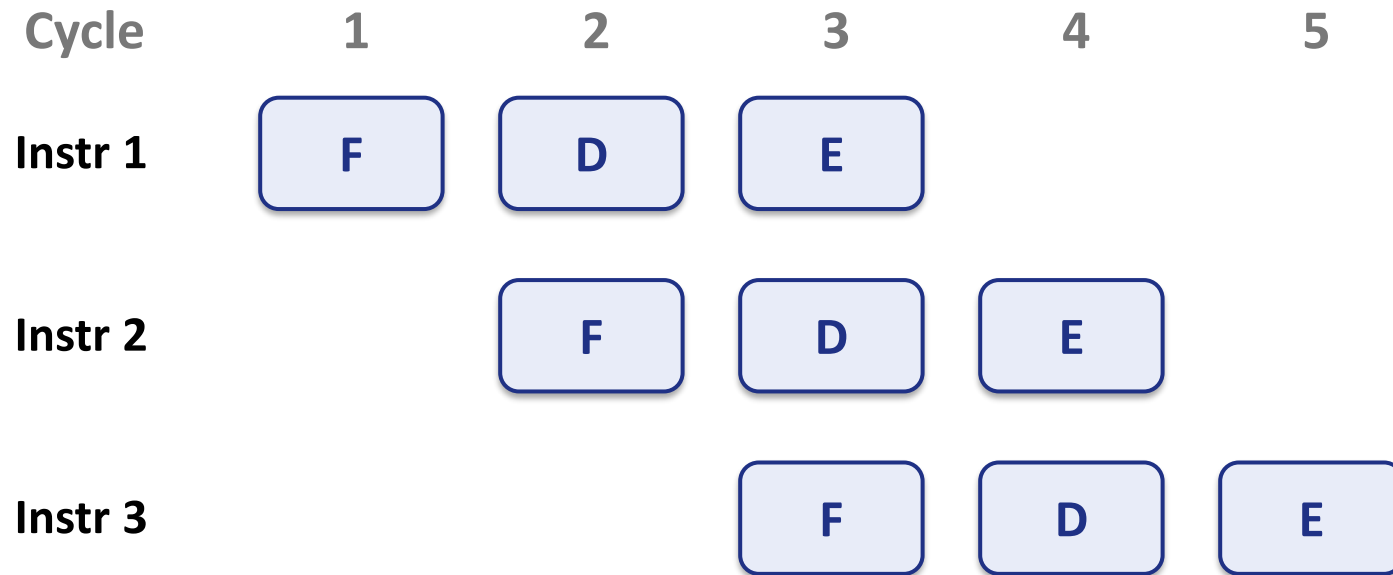
```
for j in range(m):  
    for i in range(n):  
        process(matrix[i][j])
```

Same asymptotic complexity can have very different real performance because of caches and memory access patterns.

Algorithms matter. Hardware matters too.

Modern CPUs

Pipelining



Several instructions can be at different execution stages at the same time.

Modern CPUs

Branch prediction

```
if condition:  
    A()  
else:  
    B()
```

CPU predicts a branch

Correct guess → keep work

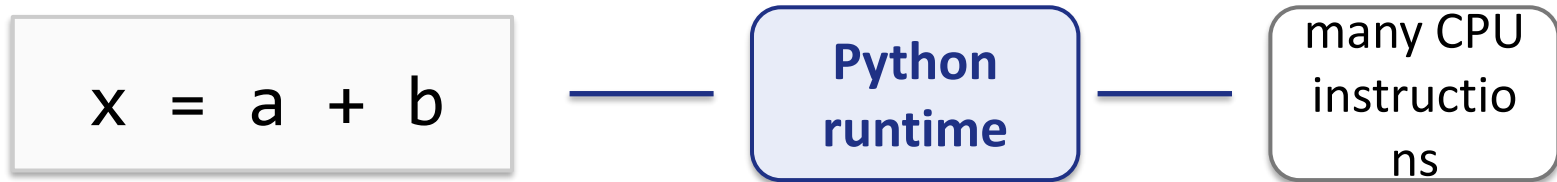
**Wrong guess → discard
work**

Modern execution is often speculative and out of order — while preserving the program's observable behavior.

Conclusion

Conclusion

One line of Python is not one CPU instruction



Between source code and hardware there may be:

bytecode

interpreter / virtual machine

compiler or JIT

runtime libraries

Conclusion

We are still missing one important layer



Who turns Python, C or Java into something the machine can execute?

Next: Programming Languages

Conclusion

What should you remember?

- Programs and data are both stored in memory.
- Memory is addressed and stores bits, not high-level variables.
- The CPU works primarily with registers, memory and simple instructions.
- An ISA is the interface between software and hardware.
- Execution follows the fetch → decode → execute cycle.
- Control flow is implemented by changing the Program Counter.
- Function calls rely on a stack of call frames.

Thank you for watching!